AIC · AI+开源 · 答辩材料

SpringGuard

基于 Open Code Review 的 Spring 智能审查增强

确定性规则 × 可选 AI × 可复现评测

方向 **AI 开发工具与开源协作**

形态 **Web + CLI + 规则包 + 评测**

效果 **Recall / Precision = 1.0**

Spring 工程里，高发缺陷总被漏掉

01 `@Transactional` 同类自调用失效；异常被吞掉，回滚语义破坏

02 Controller `@RequestBody` 缺 `@Valid`；可空链路直接 NPE

03 MyBatis 注解误用 `${}`，注入风险进入代码评审盲区

04 纯大模型：不稳定、难复核；纯人工：成本高。需要可演示、可复现的工具

一条可演示的审查流水线

INPUT

工程输入

内置样例或上传 Spring/Java ZIP（≤50MB）

- spring-buggy 正例
- spring-clean 负例
- 真实工程 ZIP

→

CORE

双路径检测

确定性规则为主，OCR / 本地模型可选

- SpringGuard 静态规则
- .opencodereview/rule.json
- merge → findings

→

OUTPUT

可核验输出

页面证据 + 金标准评测指标

- Web :5193
- Recall / Precision
- 文件与行号定位

底座开源，增强自研

UPSTREAM

Open Code Review

- alibaba/open-code-review
- 许可证：Apache-2.0
- **未修改上游源码**
- 可选接入本地 Ollama

TEAM BUILT

SpringGuard 增强层

- 确定性规则引擎
- 项目级 Spring 规则包
- 正负例 + 金标准评测
- Web 上传与证据展示

团队实际交付了什么

01

规则引擎

`spring_guard.py`

Controller / Service / Mapper 高发缺陷，秒级可复现

02

Web 服务

`app_server.py` + 前端

样例扫描 · ZIP 上传 · 严重度筛选 · 证据代码

03

评测闭环

`eval/` 金标准

正例测召回，干净样例测误报

另含规则包、编排脚本、演示视频与完整文档

可脚本复现的验证结果

1.0
召回率 Recall
金标准 6 / 6

1.0
精确率 Precision
含干净样例综合

0
干净样例误报
spring-clean

集合 / 模式	结果
spring-buggy + Guard	**TP 6/6 · Recall 1.0**
spring-clean	**0 findings · 无误报**
OCR / Hybrid	可选语义增强；本轮演示以 Guard 为主

复现：`python3 eval/run_eval.py`

不是换皮部署，是领域增强

常见做法

薄封装风险

- 直接跑通用审查 CLI
- 结果难复核、难复现
- Spring 高发点靠临场发挥
- 缺少正负例对照

本作品

可验证改进

- Spring 确定性规则可重复命中
- 证据定位到文件与行号
- 金标准 + 干净样例双验证
- 开源边界与清单完整披露

价值在可定位、可复核，而非宣称全面超过上游模型

评审可直接核验

01 公开仓库：代码 · 规则包 · 样例 · 评测 · Web · 文档

02 一键演示：`python3 scripts/app_server.py` → http://127.0.0.1:5193/

03 《开源及第三方资源使用清单》区分上游 / 自研 / 可选模型

github.com/NEX-coding/springguard

把验证做更严，把增强做更深

01

评测加强

行级匹配更严、去注释干扰，扩展真实工程负例

02

模型对照

补全 OCR 对比实验，量化语义审查真实增益

03

生态维护

更多 Spring 规则、CI 接入；视情况向上游贡献示例

— THANKS

SpringGuard

把开源 AI 审查底座，变成可复现的 Spring 工程安全网。

github.com/NEX-coding/springguard